

Development Plan Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

Table 1: Revision History

Date	Developer(s)	Change
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

This document serves as the development plan for Lake by CinqGarçons. Lake splits a large language model (LLM) across multiple devices so that they can run the model together. The plan covers how the project will be conducted. The first sections deal with confidentiality, intellectual property and licensing, followed by how the team is going to be organized for the duration of the project. Section 8 breaks down the work and lays out the schedule, Section 9 describes the proof of concept, and Section 10 lists the expected technology. The plan finishes with the use of AI tools and coding standards, while the team charter and reflections are in the appendices.

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information

There is no confidential information to protect.

2 IP to Protect

There is no intellectual property information to protect.

3 Copyright License

Lake is licensed under the GNU General Public License Version 3. A copy of the GNU GPL license is available in the Lake repository [here](#).

4 Team Meeting Plan

CinqGarçons will meet at minimum weekly every Friday at 2:40PM EST via Discord. Additional ad-hoc meetings will be scheduled as the need arises. Some ad-hoc meetings may be in-person, likely after/before lecture. There will be a minimum requirement of one in-person meeting per month. More than one meeting per month may be in-person, and it is up to the discretion of the team to decide when to convert a virtual meeting to an in-person meeting.

Each week, team members will take turns being the chair and leading the meeting. The chair will be responsible for bringing a formal agenda to each meeting. The chair will also be responsible for introducing the agenda to the team, and ensuring the agenda points are addressed during the meeting. Ad-hoc meetings may only have an informal unwritten agenda due to their potentially last-minute nature.

The order in which members will fill the role of meeting chair are as follows:

- Colin Chambachan
- Jinwoo Hong
- Matthew Nesbitt
- Gary Qin
- Ethan Walsh

5 Team Communication Plan

Formal text-based team communications will be facilitated through a Discord server, with additional communication happening verbally during meetings.

Project design and repository related communications may also occur on a pull request during the review phase, or in the comments of a related GitHub issue. All formal action items shall have an associated GitHub issue to track the item's progress. Pull requests should be associated with an issue, and may resolve said Issue. Issues will be the primary method of work assignment and work splitting within the team, as each issue shall have one or more assigned members. Issues may also be created as a discussion thread for particular design decisions that require deliberation.

6 Team Member Roles

Team members may fill roles dynamically throughout the year as best fits the situation. Some roles, however, are assigned permanent members. Members may fill more than one role, and roles may be filled by more than one member. Identified member roles are as follows:

- Liason: Ethan Walsh
- Meeting Chair: Dynamic
- Repo/Org Administrator: Matthew Nesbitt
- Technical Lead (Architecture): TBD
- Technical Lead (IPC): TBD
- Technical Lead (UI/UX): TBD
- Technical Lead (ML): TBD
- Technical Lead (Performance): TBD

7 Workflow Plan

All contributions to production are required to go through a pull request, and acquire approval from at minimum two reviewers. When working on a contribution, using either the branching workflow or forking workflow is acceptable, and it is up to the individual contributor to decide which workflow to use.

7.1 Contribution Flow

1. For branching workflow:
 - (a) Create a new working branch in the main repository. The branch name should be descriptive, and should adhere to the [conventional-branch](#) standard.
 - (b) Commit desired changes to the working branch. Commit names should be descriptive but terse, and should adhere to the [conventionalcommits](#) standard.
2. For forking workflow:
 - (a) Create a fork of the main repository. Only one fork per contributor is required.
 - (b) Create a new working branch in this fork. There are no restrictions on branch names made in forks.
 - (c) Commit desired changes to the working branch. Commit names should be descriptive but terse, and should adhere to the [conventionalcommits](#) standard.
3. Open a pull request from the working branch to the `main` branch in the main repository.
4. Address any changes requested during review.
5. After the pull request has received two approvals and passes CI, it can be merged.
6. For branching workflow:
 - (a) After a PR has been merged, the associated work branch should be deleted. This can be done directly from the PR page.

7.2 Issues

Issues will be used to track action items, and occasionally for design decisions that require deliberation. When creating an issue, a description should be included providing all the information required to address the issue. In addition, the issue may be labeled with one or more relevant tag(s) if applicable. The available tags are as follows: `bug`, `feature`, `decision`, `improvement`,

`priority: high`, `priority: low`. Pull requests should be associated with an issue, and may close said issue by including a close tag.

Issues will also be created (in separate projects) for each lecture, as well as each TA meeting. Issue templates will be created for both lecture and TA meeting issues to make the creation of these issues easier and faster.

7.3 CI/CD

CI/CD will utilize GitHub Actions, will be required to pass for a PR to be merged, and will consist of the following stages:

- PDF building and publishing
- Code linting
- Unit testing
- Integration testing
- System testing
- Performance testing

8 Project Decomposition and Scheduling

All planning runs through GitHub, and the schedule follows the course deadlines, each with an internal deadline two days earlier.

8.1 GitHub Projects

All project work is tracked on the [Lake GitHub Project](#). Every issue from the Workflow Plan gets a card on the board, and cards move through Backlog, Ready, In Progress, In Review and Done columns. An issue only moves to Ready once it has a description, a work estimate and at least one assignee. Cards are also tagged with the part of the system they touch, such as the coordinator, a node agent, networking, the dashboard or documentation.

Each deliverable in Table 2 gets its own GitHub milestone, with the due date set to the internal deadline rather than the course deadline. That way the board always reflects the date being worked toward. Lecture and TA meeting issues are kept in separate projects, as noted in the Workflow Plan, so this board only holds project work.

8.2 Breaking Down Work

Work is split by feature instead of by document. On the code side, issues map to parts of the system. On the documentation side, an issue is typically a section or one piece of analysis. Anything that looks like more than about eight hours of work gets split before anyone starts on it.

Every issue should say what done looks like. For code, that is usually a test that passes or a measurement that gets recorded. For a document section, done means the section meets the level 4 rubric criteria and has been checked against the deliverable's checklist.

The course assumes about ten hours per person per week, including class time, which gives roughly 50 hours of team time a week to plan around. At the Friday meeting, the chair goes through the board with the next internal deadline in mind. Members pick up Ready issues, and the chair makes sure nobody is overloaded while someone else has nothing to do. Members shall also take issues outside their own technical lead area now and then, so that more than one person understands each component.

8.3 Schedule

The team's schedule is built around the given course deadlines while allocating an internal deadline two days before each public facing one. Reserving the last two days allows for double-checking deliverables and catching issues that slip through. For demonstrations such as the proof of concept (PoC), the internal deadline is two days before the assigned slot, and after that point only bug fixes are merged. Table 2 lists every graded deliverable, with dates taken from the course calendar. Smaller items like the contribution surveys, the productivity reports and the module implementation exercise go on the board with the same two-day buffer.

Implementation runs alongside the documents in the phases shown in Table 3.

Table 2: Course deliverables with official and internal deadlines

Deliverable	Weight	Official	Internal
<i>Fall term, 2026</i>			
Problem Statement, PoC Plan, Development Plan and Team Charter	2%	Sept 28	—*
Peer review of another team’s Problem Statement and Development Plan	—	Oct 1	Sept 29
SRS and Hazard Analysis (Rev 0)	7%	Oct 19	Oct 17
Peer review of SRS and Hazard Analysis	—	Oct 22	Oct 20
V&V Plan (Rev 0)	5%	Nov 2	Oct 31
Peer review of V&V Plan	—	Nov 5	Nov 3
Design Document (Rev −1) and Design Thinking Report (CD-Fall)	3%	Nov 16	Nov 14
Peer review of Design Document (Rev −1)	—	Nov 19	Nov 17
PoC Demonstration and interviews	5%	Nov 23 to Dec 3	†
<i>Winter term, 2027</i>			
Design Document (Rev 0)	4%	Jan 18	Jan 16
Revision 0 Demonstration and interviews	10%	Feb 1 to 11	†
V&V Report (Rev 0) and Performance Report (CD-Winter)	4%	Mar 8	Mar 6
Peer review of V&V Report and CD-Winter	—	Mar 11	Mar 9
Final Demonstration (Rev 1)	20%	Mar 15 to 21	†
Final Documentation (Rev 1), EXPO poster and video	30%	Mar 31	Mar 29
Capstone EXPO	10%	Apr 6 (TBC)	Apr 4

*The internal deadline policy was adopted while this deliverable was already in progress.

†Two days before the team’s assigned slot.

Table 3: Implementation phases

Phase	Dates	Main work
Setup	Sept 28 to Oct 16	Get ggml and llama.cpp building on every team member’s machine and record single-device baselines
Proof of concept	Oct 19 to the PoC slot	The two-device split described in Section 9
Core system	December to the Rev 0 slot	Any number of devices, profiling, the partitioner, node agents joining and leaving, the dashboard, and re-splitting the model between requests
Dynamic allocation	Feb 12 to Mar 5	Re-splitting during a session, benchmarks for the Performance Report, and the wireless stretch goal
Refinement	Mar 8 to Apr 6	Feedback from Rev 0, final documentation and EXPO preparation

9 Proof of Concept Demonstration Plan

The PoC is meant to show early on that the project is feasible and to establish the scope of what shall be delivered by the end of the project.

9.1 Risks

The biggest risk is not being able to split a model across devices at all. Tools like llama.cpp run the whole model in one process, so splitting it means working directly with ggml, the tensor library underneath. Each device has to load only its own range of layers from the model file, run them, keep its own key-value (KV) cache, and pass the hidden state on to the next device intact. Every other part of Lake depends on this working.

Two smaller risks come along with it. The first is network overhead. Text is generated one token at a time, and each token has to cross every device connection. One transfer is small, but its latency is added to every token. On a slow link, a split model could end up noticeably slower than running it on one machine.

The second risk is mixing platforms. Kernels differ across device types, so their floating-point results are not bit-identical. Devices on different platforms have to agree on the hidden state format, and the output has to stay coherent despite the small numerical differences.

Dynamic re-allocation is what sets Lake apart from existing tools, but it is left out of the PoC. Re-allocation cannot work until the basic split does, and

trying to do both by November would turn the PoC into a prototype.

The PoC is also not expected to be faster than a single device. With a pipeline split, each token still goes through every layer in order, so splitting a model that already fits on one machine does not speed up a single request. Splitting pays off when a model is too big for any one device, or when several requests run at once. For the PoC, the goal is to measure how much overhead the split adds.

9.2 Demonstration

The demonstration uses a model (which is to be determined) to be split by hand down the middle. Initially, a model will be split up and run on two separate processes on one system to prove feasibility, and it will be further iterated on if feasible. A bare-bones command-line coordinator hands out the layer ranges. The partitioner, profiler and dashboard are not part of the PoC.

The demonstration builds up in three steps, with each step adding one more thing that can go wrong:

1. Two processes on the same laptop, talking over localhost. This tests the split by itself, without a real network or different platforms.
2. Two laptops of the same type connected by wired Ethernet, which adds real network latency.
3. Two laptops with different operating systems and processor architectures, for example an Apple Silicon MacBook and an x86 Windows or Linux machine.

Each step runs a prompt from the command line and compares the streamed output with the same prompt run through plain llama.cpp on one machine. Each device's memory use is also shown, to confirm that it loaded only its own layers, along with a per-token breakdown of compute time on each device against time spent on transfers.

The PoC counts as a success if the split model produces a sensible response to a prompt, with the hidden state passed between the two halves without errors. Reaching this in step one is enough. Steps two and three are attempted if time allows, and any progress on them is shown as extra evidence. The output does not need to match a single device exactly, and there is no speed target. Transfer times are recorded to guide later work, but they do not decide whether the PoC passes.

9.3 Fallback Options

If the PoC shows that one of these risks is worse than expected, the scope will be discussed with the TA afterward. The current fallback ideas are:

- If a custom split on top of ggml cannot be made to work, build Lake’s coordination and allocation on llama.cpp’s remote procedure call (RPC) backend instead.
- If network overhead is too high, move the performance goal toward models that do not fit on one device and toward several requests at once, since splitting still helps there.
- If splits across platforms produce bad output, limit Rev 0 to the platform combinations that work and make the rest a stretch goal.

10 Expected Technology

These are the current technology choices, which are subject to change once building starts. The SRS and design documents will not depend on them. Alongside the tools below, git and GitHub are used for version control, issues, pull requests, the project board and document publishing, and Discord for day-to-day communication.

10.1 Languages and Frameworks

The coordinator will be written in Python. Its job is handing out layer assignments and collecting measurements, so it is not on the per-token path and Python’s speed is not a concern there. Go and Rust were discussed, but the extra speed would not help in that part of the system, and the whole team is already comfortable with Python. The client-facing application programming interface (API) will use FastAPI, and the dashboard will be built with React and TypeScript.

The inference engine on each device is expected to be written in Rust. The ggml library is written in C, and splitting the model means building ggml’s compute graph directly. Rust matches C++ in speed and calls C functions directly, so there is no extra layer on every token. Rust also enforces memory safety and resource cleanup at compile time, where C++ leaves both to the programmer, which makes it easier to learn for a team that has mostly not worked in C++. The calls into ggml still need unsafe Rust, so that code is kept in one small module. C++ was the other option considered, since llama.cpp is written in it. If Android support happens, the node agent there would be written in Kotlin and call the same Rust code through the Java Native Interface (JNI).

Devices talk to each other and to the coordinator using gRPC with Protocol Buffers. From one message definition, matching Python and Rust code is generated, using the Tonic library on the Rust side, and gRPC’s streaming calls fit the token-by-token handoff. The downside is some framing overhead compared with raw sockets, which the PoC will measure. If the overhead turns out to matter, the alternatives to evaluate are raw TCP with FlatBuffers, ZeroMQ and QUIC, and QUIC would also help on unreliable Wi-Fi.

10.2 Inference and Models

The model math itself will run on ggml, the tensor library underneath llama.cpp. The library already has fast quantized kernels for both x86 and ARM, plus GPU backends such as Metal and CUDA, which covers every target platform. Lake’s own code handles the splitting, networking and profiling around it. The catch is that ggml’s API is low-level and changes often, so ggml will be pinned to a specific version.

Another option considered was ONNX with ONNX Runtime. ONNX models are portable, and standard tools can cut an ONNX graph into subgraphs, which would suit layer splitting. The deciding factor for ggml was that quantized versions of popular open models are much more widely available as GGUF files. ONNX Runtime stays as the backup if ggml’s API proves too hard to work with.

Lake is meant to be model agnostic. Any pre-trained transformer LLM that ggml supports should work, rather than one specific model, and no model will be trained or fine-tuned. Small models are used during development because they load quickly, and larger ones are used later to test running a model that does not fit on one device. Weights are stored as 4-bit quantized GGUF files, which are roughly a third of the size of 16-bit weights. Each device memory-maps the file and reads only its own layers. Weights are downloaded under each model’s own license and are not committed to the repository.

10.3 Components Built for Lake

Existing projects such as `exo`, `prima.cpp` and `llama.cpp`’s RPC backend can already split models across machines, and they will serve as references. Lake still needs its own coordinator, partitioner and transport between devices. In `exo`, layers are assigned once at the start of a session, and `llama.cpp`’s RPC backend fixes the split when the model loads. Changing the split during a session, which is Lake’s dynamic allocation goal, needs control over layer assignment and the KV cache that these tools do not offer.

The partitioner will profile each device’s per-layer speed, free memory and network latency, then pick the split points that give the lowest time per token. A simple greedy approach comes first, with dynamic programming replacing it once that works.

10.4 Testing and Continuous Integration

Table 4 shows the linters and unit testing frameworks for each language. Code coverage on the Python side will use `pytest-cov`. On the Rust side, `cargo-llvm-cov` and `tarpaulin` both still need to be tried with the build before picking one. A coverage target will be set during the PoC phase, once there is enough code to measure.

GitHub Actions already builds and publishes the $\text{\LaTeX}$ documents, and a second workflow for code will run the stages listed in the Workflow Plan on

Table 4: Linters and unit testing frameworks by language

Purpose	Python	Rust	TypeScript
Linting and formatting	Ruff	Clippy, rustfmt	ESLint, Prettier
Unit testing	pytest	cargo test	Vitest

every pull request. The Rust build and tests will run on Linux, macOS and Windows runners, so anything platform-specific breaks right away rather than at a demo. For integration tests, continuous integration (CI) will split a tiny test model across two local processes and compare the result with an unsplit run, since a real model would take too long on every pull request. CI runners are shared virtual machines with noisy timing, so the performance stage there is only a smoke test for large regressions. Real performance numbers and system tests will come from team members’ own machines before each demo.

10.5 Performance Measurement

Most performance data will come from Lake’s own per-token profiler, which records compute time on each device and transfer time between devices. The same data feeds the partitioner and the dashboard. For comparison, llama-bench and llama-perplexity from llama.cpp give single-device numbers for speed and output quality. The unsafe code that calls into ggml will also be tested under AddressSanitizer in CI to catch memory bugs early.

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

[What coding standard will you adopt? —SS]

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Appendix — Team Charter

[borrows from University of Portland Team Charter —SS]

External Goals

[What are your team’s external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Attendance

Expectations

[What are your team’s expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

[What are your team’s expectations regarding the quality of team members’ preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team’s expectations regarding team members’ ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]